ADITYA UNIVERSITY

COA UNIT-4
Memory Organization
&
I/O Organization

ADITYA UNIVERSITY

UNIT-4
PART-A: Memory Organization

- Memory Hierarchy
- Main memory
- Auxiliary memory
- Associate Memory
- Cache Memory
- Virtual memory

Memory Hierarchy

- Memory hierarchy is the **organization of different types of memory** in a computer system based on **speed, cost, and capacity**.
- **Levels of Memory**
  - ✓ **Cache Memory:**

    Very fast memory located close to the CPU. Stores frequently used data.
  - ✓ **Main Memory (RAM):**

    Stores currently running programs and data. Slower than cache but larger.
  - ✓ **Auxiliary Memory:**

    Permanent storage with large capacity (Magnetic Disks, Magnetic Tapes). Slowest.

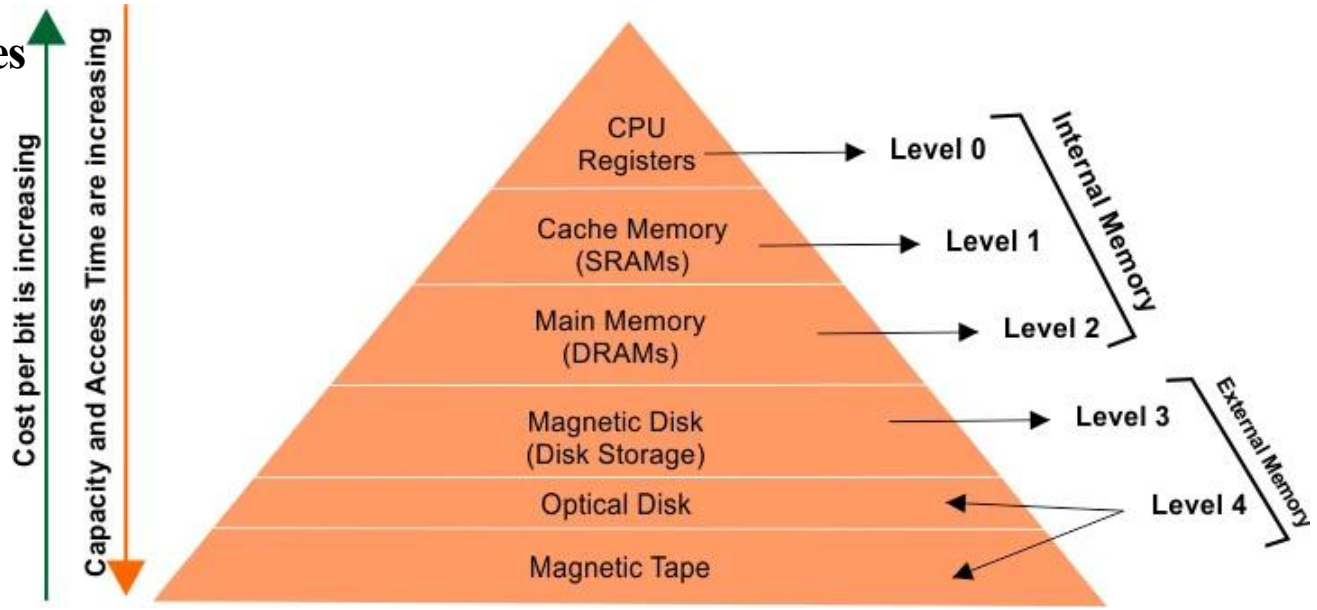


Fig. Memory hierarchy in a computer system

Memory Hierarchy

- **CPU ↔ Cache Memory:**

  CPU first checks cache for required data (fast access).

- **Cache Memory ↔ Main Memory:**

  If data is not in cache, it is fetched from main me

- **CPU ↔ Main Memory:**

  CPU can also directly communicate with main memory when needed.

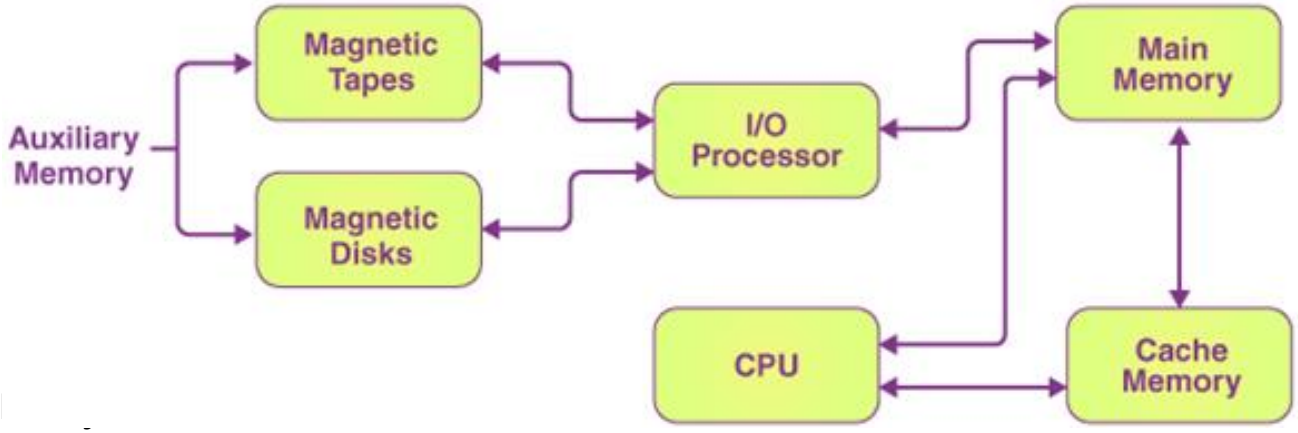


Fig. Data Flow and Interconnection in Memory Hierarchy

- **Main Memory ↔ I/O Processor:**

  I/O processor transfers data between main memory and external devices.

- **I/O Processor ↔ Auxiliary Memory:**

  Magnetic disks and tapes are accessed through the I/O processor

Main Memory

- Main memory is the **central storage unit** of a computer used to store programs and data during execution.
  - It is **directly accessed by the CPU**.
  - It stores **currently running programs and data**.
  - It is a **temporary memory** (data is lost when power is OFF).
  - It is **faster than secondary memory** but slower than cache.
- **Types of Main Memory**

1. **RAM (Random Access Memory)**
   - **Static Ram**
   - **Dynamic Ram**
2. **ROM (Read Only Memory)**

RAM (Random Access Memory)

- **RAM (Random Access Memory)** is a fast, volatile memory that temporarily stores data and instructions currently in use by the CPU to enable quick access and efficient processing. RAM is volatile memory, which means that the data stored in it is lost when the power is turned off. It is also called read-write memory or the main memory or the primary memory.
- **Types of Random Access Memory (RAM):**
  - Static RAM (SRAM)
  - Dynamic RAM (DRAM)

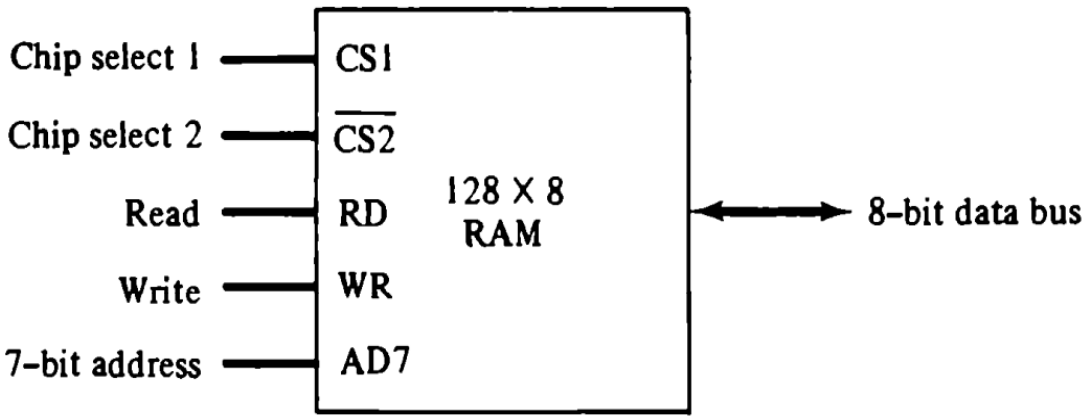
Static RAM: SRAM stands for Static Random Access Memory. It is a type of semiconductor which is widely used in computing devices and microprocessors.

Dynamic RAM: DRAM stands for Dynamic Random Access Memory. It is made of Capacitors and has smaller data life span than Static RAM.

Typical RAM (128 × 8)

- **RAM** is a memory used to **store temporary data** (data is lost when power is OFF).
- **128 × 8 means:** 128 memory locations, each storing 8 bits (1 byte).
- **Address lines (7 lines):** Used to select one location out of 128.
- **Chip Select (CS1, CS2):** Used to enable the RAM chip; if not selected, no operation occurs.
- **RD (Read):** Used to read data from RAM.
- **WR (Write):** Used to write data into RAM.
- **Data Bus (8-bit):** Transfers data to and from RAM (bidirectional).
- **Inhibit (No Operation):** When the chip is not enabled, RAM does not respond and the data bus is in **high-impedance** state.
- **Write Operation:** When chip is enabled and **WR = 1**, data from the data bus is stored into the selected memory location.
- **Read Operation:** When chip is enabled and **RD = 1**, data from the selected memory location is sent to the data bus.

Typical RAM chip



(a) Block diagram

CS1	$\overline{CS2}$	RD	WR	Memory function	State of data bus
0	0	×	×	Inhibit	High-impedance
0	1	×	×	Inhibit	High-impedance
1	0	0	0	Inhibit	High-impedance
1	0	0	1	Write	Input data to RAM
1	0	1	×	Read	Output data from RAM
1	1	×	×	Inhibit	High-impedance

(b) Function table

Read Only Memory(ROM)

ROM (Read-Only Memory) is non-volatile memory that permanently stores essential instructions and data needed to start and operate a computer. Types: MROM(Masked ROM), PROM (Programmable ROM), EPROM (Erasable PROM), EEPROM.

1. Mask ROM (MROM)

- Permanently programmed during manufacturing.
- Cannot be modified after production.
- Used in firmware for devices like keyboards, BIOS chips (older systems), and embedded systems.

2. PROM (Programmable ROM)

- One-time programmable (OTP) by the user using a special device (PROM programmer).
- Fuses or anti-fuses are burned to store data permanently.
- Used in prototypes and low-volume production.

erasure.

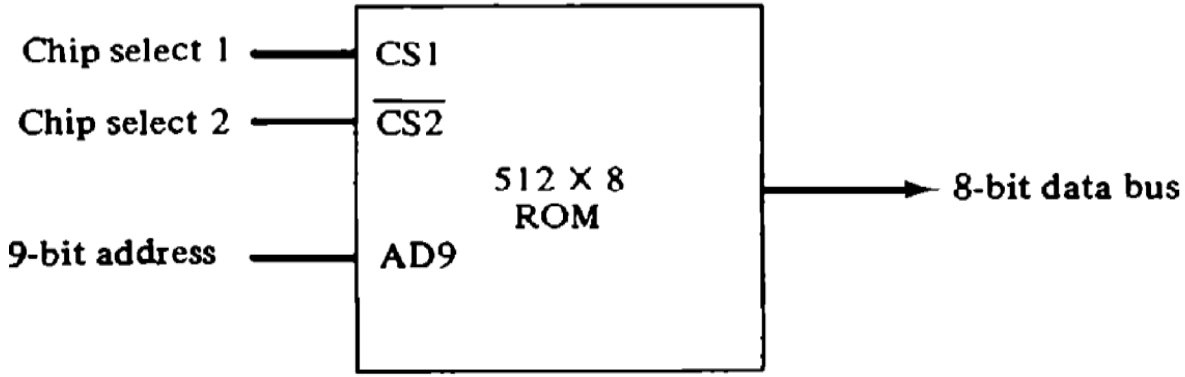
3. EPROM (Erasable Programmable ROM)

- Can be erased and reprogrammed using UV light.
- Features a quartz window for UV exposure.
- Used in firmware development and testing.

Typical 512 X 8 ROM

- **ROM** is a memory used to store **permanent data** (non-volatile).
- **512 × 8 means:** 512 memory locations, each storing 8 bits (1 byte).
- **Address lines (9 lines):** Used to select one location out of 512.
- **Chip Select (CS1, CS2):** Used to enable the chip; if not selected, no output is given.
- **Data Bus (8-bit):** Transfers 8-bit data from ROM to CPU.
- **Working:** CPU gives an address → chip is enabled → ROM provides the the stored data to CPU through data bus.
- **Only read operation is possible** (no writing).

Typical ROM chip



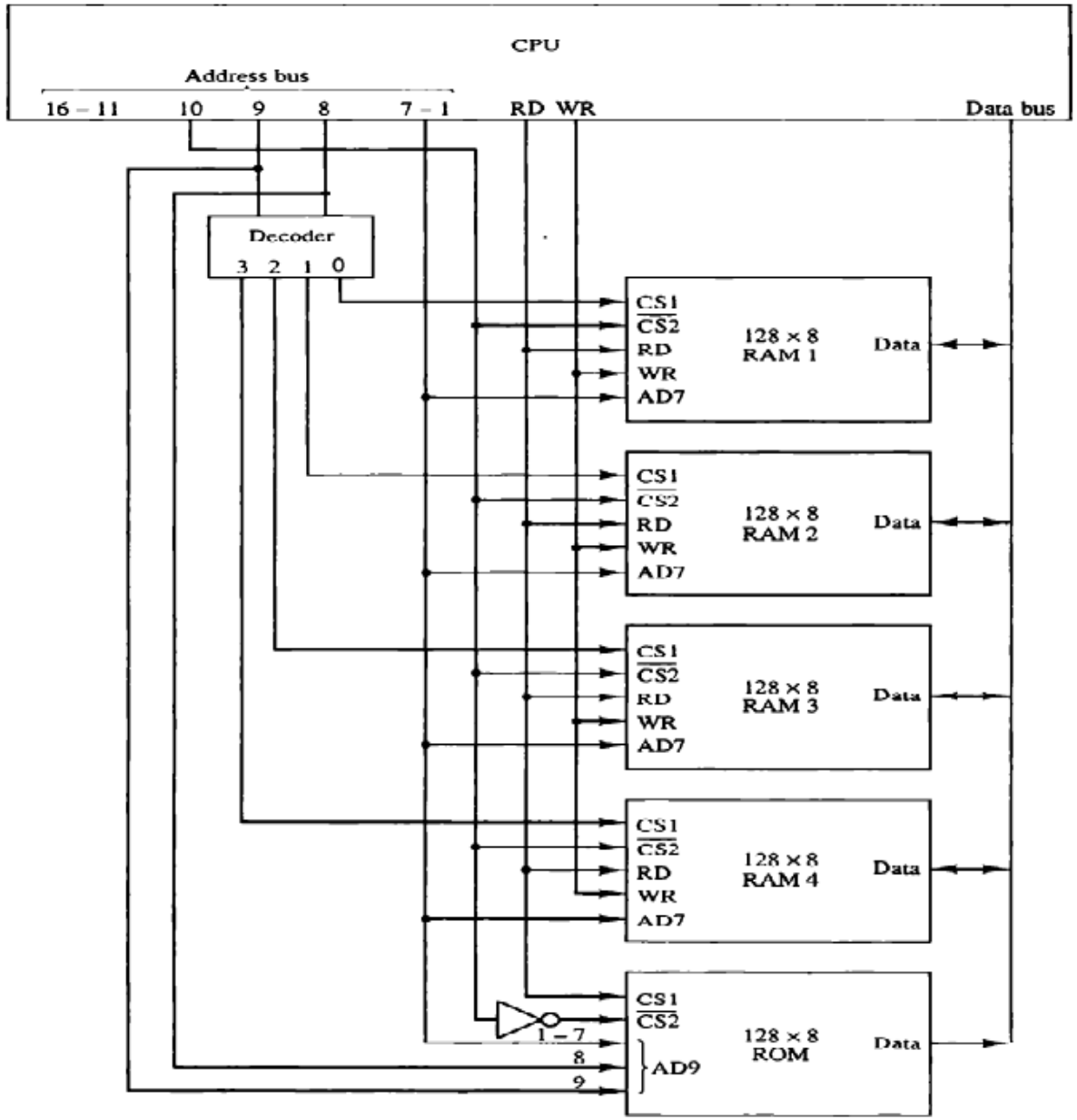
ADITYA UNIVERSITY

Comparison of RAM and ROM

Feature	RAM (Random Access Memory)	ROM (Read-Only Memory)
Volatility	**Volatile** – data is lost when power is turned off	**Non-volatile** – retains data even when power is off
Type of Memory	Temporary memory	Permanent memory
Primary Use	Stores data and instructions currently in use	Stores firmware and boot instructions
Read/Write Capability	Can be read and written frequently	Usually only readable. some types allow limited writing
Function	Used by CPU to run programs and active tasks	Used to start the system and load essential programs
Speed	Fast read/write speed	Slower compared to RAM
Size in System	Larger – typically in gigabytes (GB)	Smaller – usually in megabytes (MB)
Data Change Frequency	Data changes constantly during operation	Data remains fixed or rarely updated
Types	DRAM (Dynamic RAM), SRAM (Static RAM)	PROM, EPROM, EEPROM

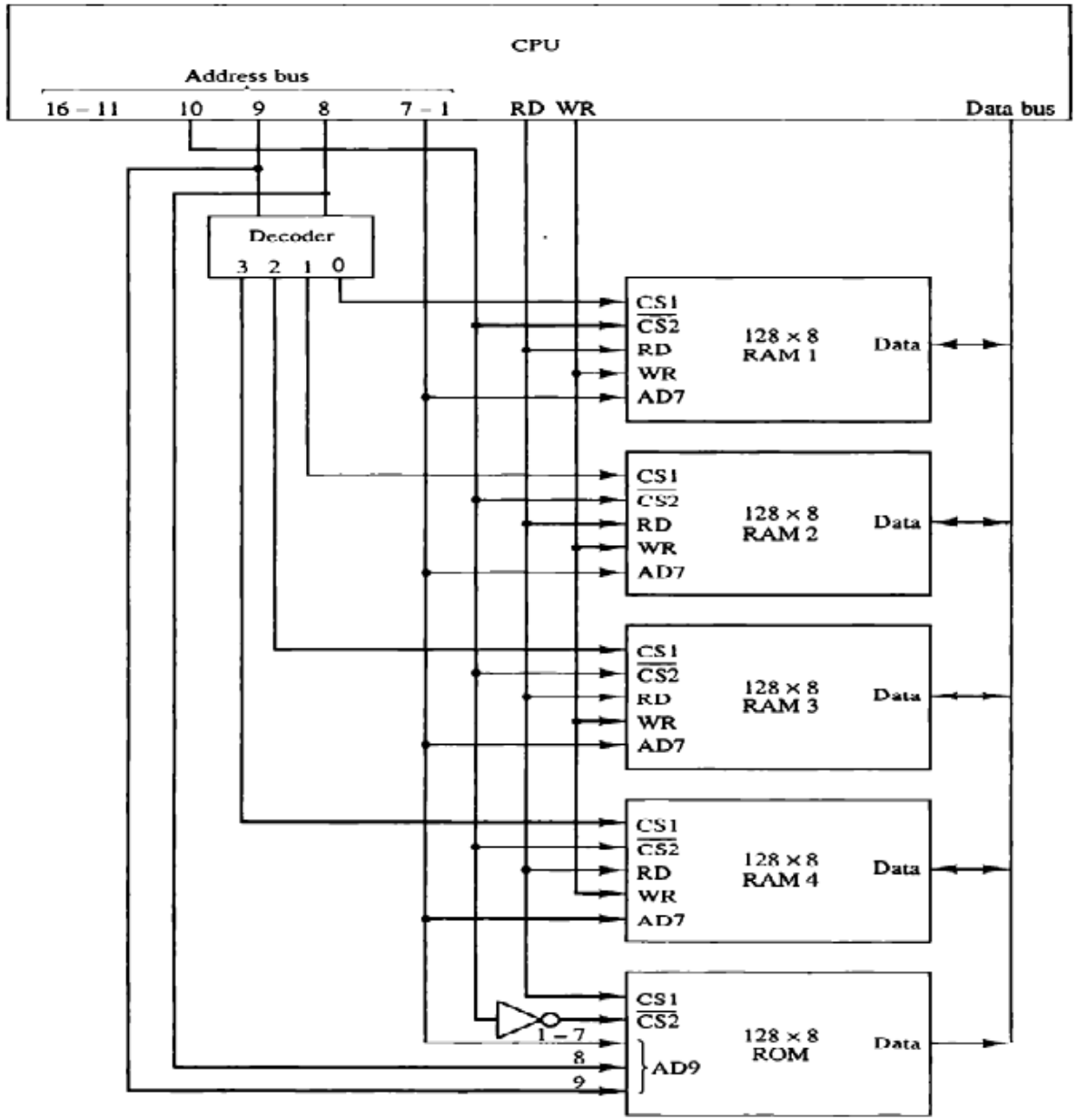
Connection of Memory To CPU

- RAM and ROM chips are connected to memory using three main buses:
  - **Address Bus** → selects memory location
  - **Data Bus** → transfers data
  - **Control Signals (RD, WR)** → control read/write operations
- The address lines are divided:
  - Higher-order bits (e.g., 9, 8) go to a decoder
  - Lower-order bits (e.g., 7–1) go directly to AD7 of memory chips
- The Decoder activates one memory chip at a time using Chip Select (CS) signals.



Connection of Memory To CPU

- There are:
  - **4 RAM chips (128 × 8 each)** → used for read and write operations
  - **1 ROM chip (128 × 8)** → used only for read operations
- **RD (Read)** signal: CPU reads data from selected memory chip
- **WR (Write)** signal: CPU writes data into selected RAM chip
- Only **one memory chip is active at a time**, based on decoder output.
- The **data bus is common**, so all chips share it, but only the selected chip sends/receives data.



MEMORY ADDRESS MAP

- The Memory Address Map shown in Table, which tells the CPU, which part of memory (RAM or ROM) is selected for a given address.
- The CPU uses the address bus to select a memory location.
- Each memory chip (RAM or ROM) is assigned a specific range of addresses.
- Based on the address, the system decides which memory chip should respond.

TABLE 12-1 Memory Address Map for Microprocomputer

Component	Hexadecimal address	Address bus 10	9	8	7	6	5	4	3	2	1
RAM 1	0000–007F	0	0	0	x	x	x	x	x	x	x
RAM 2	0080–00FF	0	0	1	x	x	x	x	x	x	x
RAM 3	0100–017F	0	1	0	x	x	x	x	x	x	x
RAM 4	0180–01FF	0	1	1	x	x	x	x	x	x	x
ROM	0200–03FF	1	x	x	x	x	x	x	x	x	x

- If the CPU sends an address between 0000 and 007F, only RAM 1 is selected.
- If the address is between 0200 and 03FF, the ROM is selected.
- **Address Bus Bits (10 to 1):**
  - The address bus is divided into: Higher bits (10, 9, 8) → used for selecting the memory chip
  - Lower bits (7 to 1) → used for selecting location inside the chip

Difference Between Static RAM and Dynamic RAM

Feature	Static RAM (SRAM)	Dynamic RAM (DRAM)
Technology	Uses flip-flops to store data	Uses capacitors to store data as an electrical charge
Refresh	No need of Refreshing	Requires periodic Refreshing to retain data
Speed	Faster access time	Slower access time
Cost	More expensive	Less expensive
Power Consumption	Higher power consumption	Lower power consumption
Density	Lower density (less storage per chip)	Higher density (more storage per chip)
Typical Use	Cache memory and registers	Main memory (RAM) in computers and other devices

ADITYA UNIVERSITY

AUXILIARY MEMORY

- Auxiliary memory is also called as Secondary Memory.
- It is used to store large amount of data.
- It is Non-volatile.
- It can store tera-bytes of data.
- It is cheaper as compared to primary memory.
- Based on Access, Auxiliary memory is classified into 2 types:
  - Sequential Access Device
  - Random Access Device
  - Example for Sequential Access Device is Magnetic Tape.
  - Example for Random Access Device is Magnetic Disk.

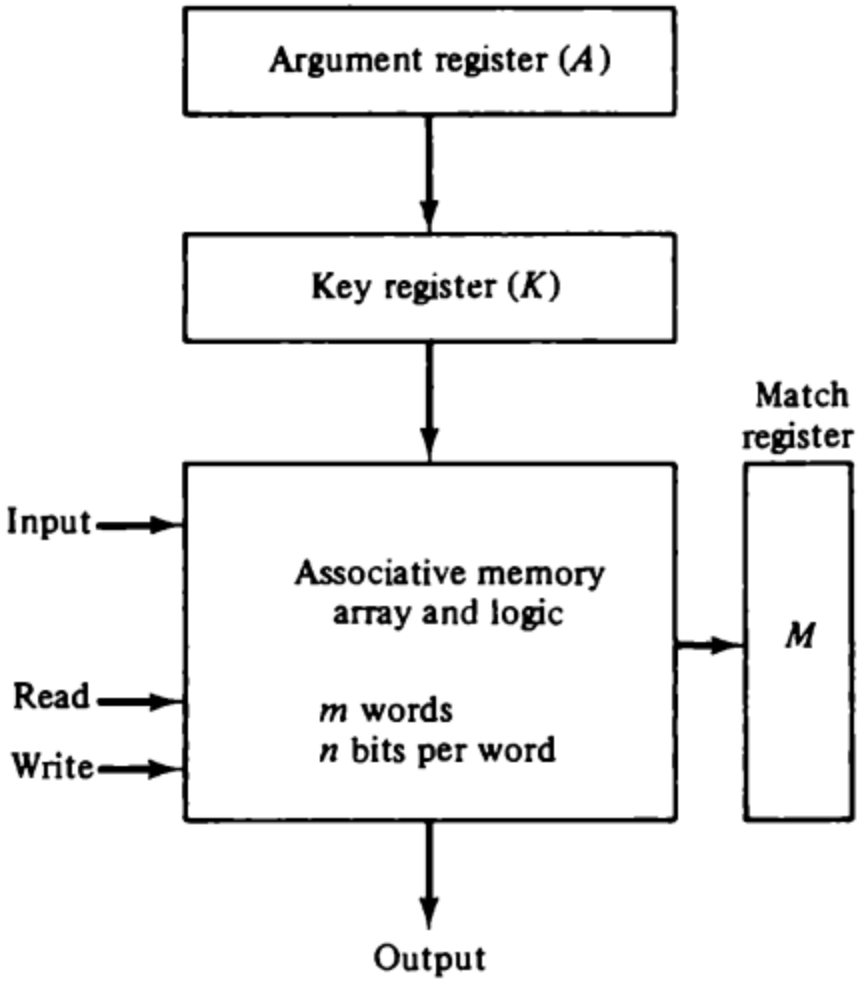
ADITYA UNIVERSITY

Comparison of Magnetic Disk and Magnetic Tape

S.No	Feature	Magnetic Disk	Magnetic Tape
1	**Access Type**	Random access	Sequential access
2	**Speed**	Fast data access	Slow (must scan sequentially)
3	**Data Retrieval**	Directly any location	Must search from beginning
4	**Usage**	Online storage (HDD, SSD systems)	Backup and archival storage
5	**Cost**	More expensive	Cheaper
6	**Storage Capacity**	High	Very high (suitable for bulk data)
7	**Data Transfer Rate**	High	Low
8	**Update/Modify Data**	Easy	Difficult
9	**Reliability**	Good for frequent use	Better for long-term storage
10	**Suitability**	Suitable for fast and frequent access	Suitable for low-cost, long-term storage and backups
11	**Examples**	Hard Disk, Floppy Disk	Magnetic Tape (Tape Drive)

Associate Memory

- A memory unit accessed by content is called an associative memory or content-addressable memory (CAM).
- CAM is accessed based on data content rather than by specific address or location.
- Parts of Associate Memory:
  - **Argument Register (A):** Holds the data to be searched.
  - **Key Register (K):** Decides which bits to compare (mask).
  - **Memory Array:** Stores data (m words, n bits each) and compares all words simultaneously.
  - **Match Register (M):** Indicates matching words (1 = match, 0 = no match).
  - **Input/Output & Control:** Input writes data, output gives result, and Read/Write signals control operation.
- **Working Process:**
  - Load the search data into Argument Register (A)
  - Set Key Register (K) to select bits for comparison
  - Memory compares all stored words in parallel
  - Matching words are indicated in Match Register (M)
  - Output gives the matched data



Block Diagram of Associative Memory

CACHE MEMORY

- **Concept of Cache Memory**
  - Cache is a **small, fast memory** placed between CPU and main memory.
  - It stores **frequently used data and instructions**.
  - Access time is much lower than main memory.
  - Improves overall system performance.
  - Cache access time is about **5 to 10 times faster** than main memory.
- **Working of Cache**
  - CPU first checks cache:
    If found → **Cache Hit**
    If not found → **Cache Miss**
  - On miss:
    Data is fetched from main memory and stored in cache for future use
- **Hit Ratio** = (Number of Hits) / (Total Memory Accesses)
- High hit ratio → better performance

Cache Memory - Locality of Reference

- Programs tend to access **same memory locations repeatedly** within a short time. This is called **locality of reference**.
- Types:
  - **Temporal locality** → recently used data is reused
  - **Spatial locality** → nearby memory locations are accessed

Cache Memory Mapping/Mapping Functions

- Memory mapping is the method used to **decide how the data placed into cache memory from main memory.**
- Cache is much smaller than main memory, so we need a rule to decide **where to store a particular memory block in cache** . This rule is called a **mapping function or mapping technique.**
- **There are three types of Memory Mapping Techniques:**
  - ✓ **Direct Mapping**

    Each main memory block maps to **one fixed cache location**

    Simple but may cause conflicts
  - ✓ **Associative Mapping**

    A block can be placed **anywhere in cache**

    Flexible but costly
  - ✓ **Set-Associative Mapping**

    Combines both methods

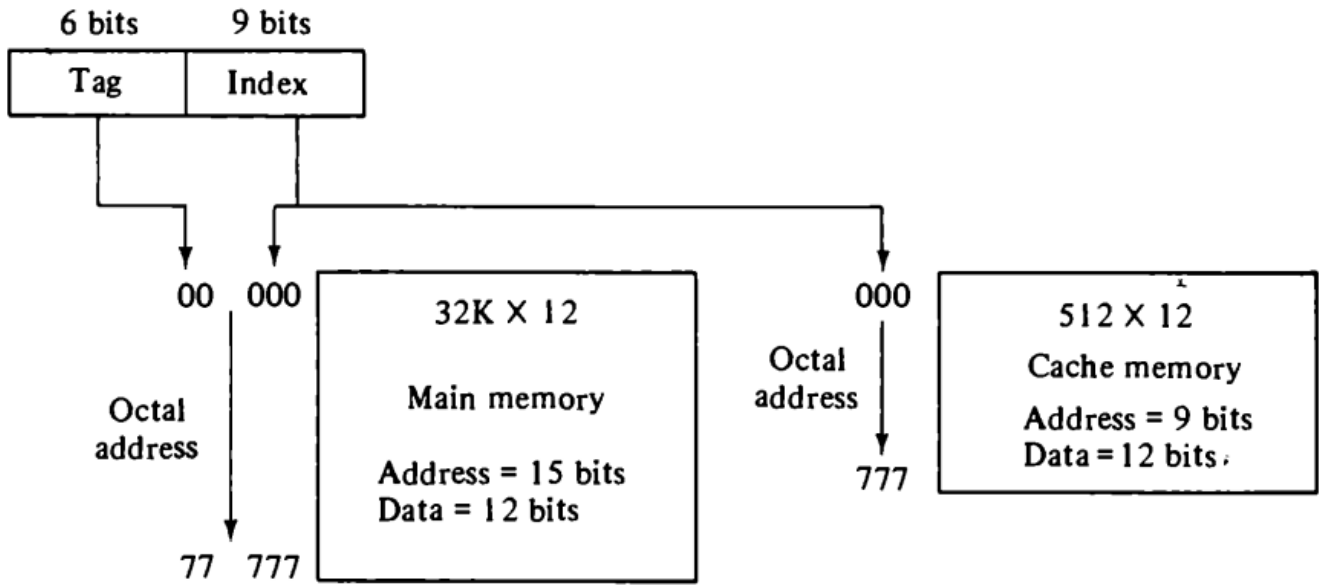
    Cache divided into sets; block can go to **any location within a set**

Cache Memory - Direct Mapping

Direct Mapping with Example:

- Each main memory block maps to **one fixed cache location**
- Main Memory size = **32K × 12** → needs **15-bit address**
- Cache size = **512 × 12** → needs **9-bit address**
- So, the **15-bit address is divided into: Index (9 bits)** → selects cache line,
  Tag (6 bits) → identifies which block is stored

Figure 12-12 Addressing relationships between main and cache memories.



Cache Memory - Direct Mapping - Example

Step 1: CPU generates address (In octal format)

Example: Address = **00000**

- Split into: **Tag → 00** & **Index → 000**

Step 2: Index selects cache location

- Index **000** → points to **first cache line**

Step 3: Tag comparison

- Cache line stores:
  - Data = **1220** & Tag = **00**
- Compare:
  - If **tag matches → Cache Hit**
  - If **not match → Cache Miss**
- **For example, from the diagram:**

Address **00000** → Data **1220** is stored in cache index **000**, tag **00**

Address **02777** → Data **6710** is stored in cache index **777**, tag **02**

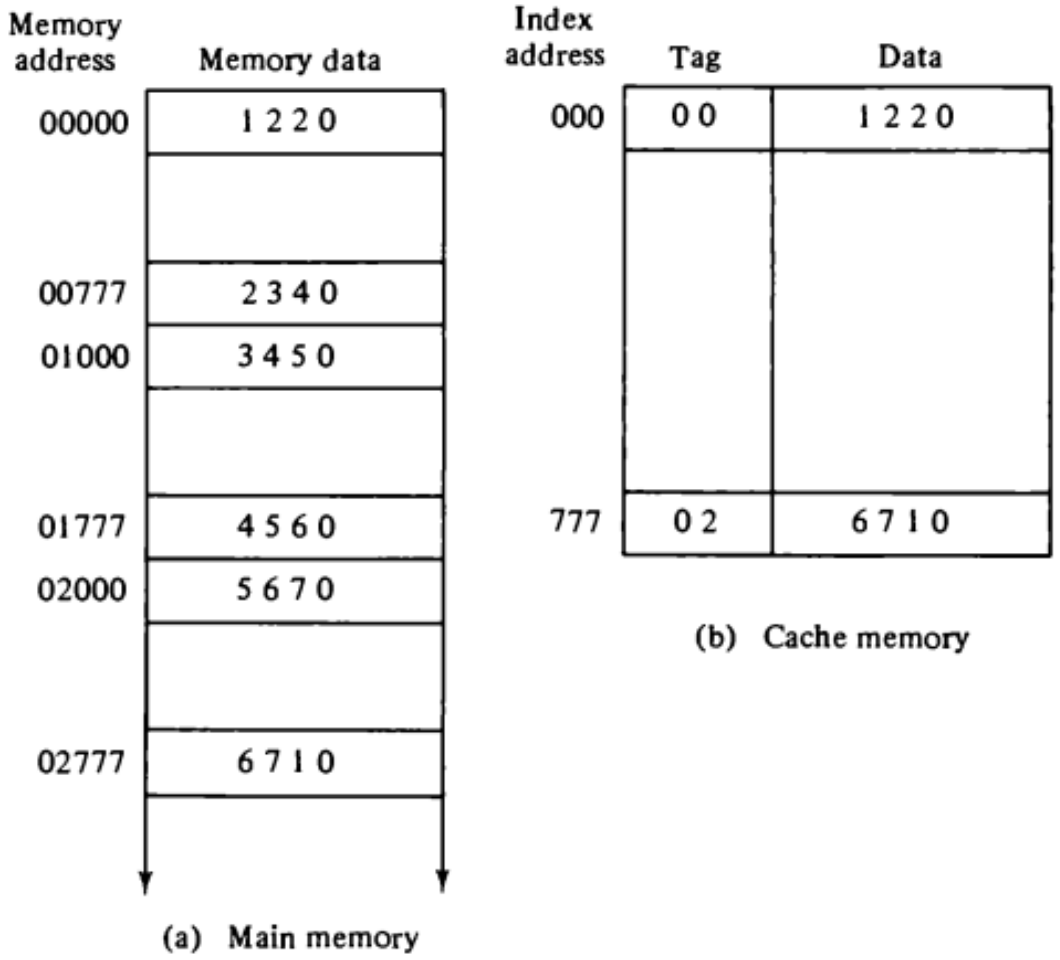


Figure: Direct Mapping Cache Organization

Cache Memory - Associate Mapping - Example

- In **associative mapping, any main memory block can be stored in any location of cache**.
  There is **no fixed position** like in direct mapping.
- CPU gives a **15-bit address** (In Octal format)
- This address is loaded into **Argument Register**
- Example entries in cache:
  - Address **01000 → Data 3450**
  - Address **02777 → Data 6710**
  - Address **22345 → Data 1234**
- **How it Works**
- CPU sends an address (say **02777**)
- This address is compared with **all stored addresses in cache simultaneously**
- If a match is found: **Cache Hit** → data (6710) is given
- If no match: **Cache Miss** → fetch from main memory

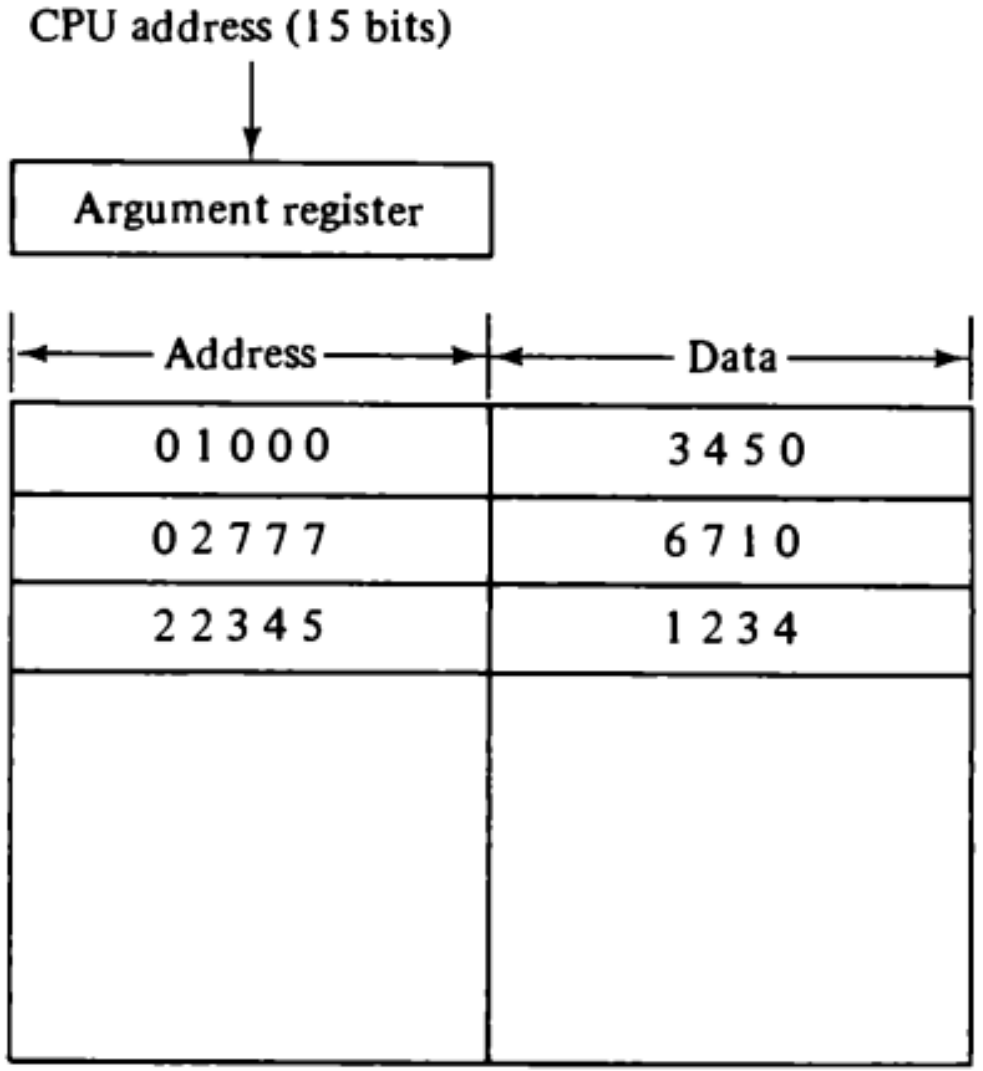


Fig. Associate Mapping Cache Organization

Cache Memory – Set-Associate Mapping - Example

- In Set-Associative Mapping, cache is divided into sets, and each set has multiple locations (ways).
- A memory block maps to one set, but can be placed anywhere within that set.
- Cache is divided into sets (Index = 000 to 777)
- Each set has 2 blocks (2-way set associative)
- **Example 1:** At index **000**, there are **two positions**:

Tag	Data
01	3450
02	5670

- **Example 2:** At index **777**, there are **two positions**:

Tag	Data
02	6710
02	2340

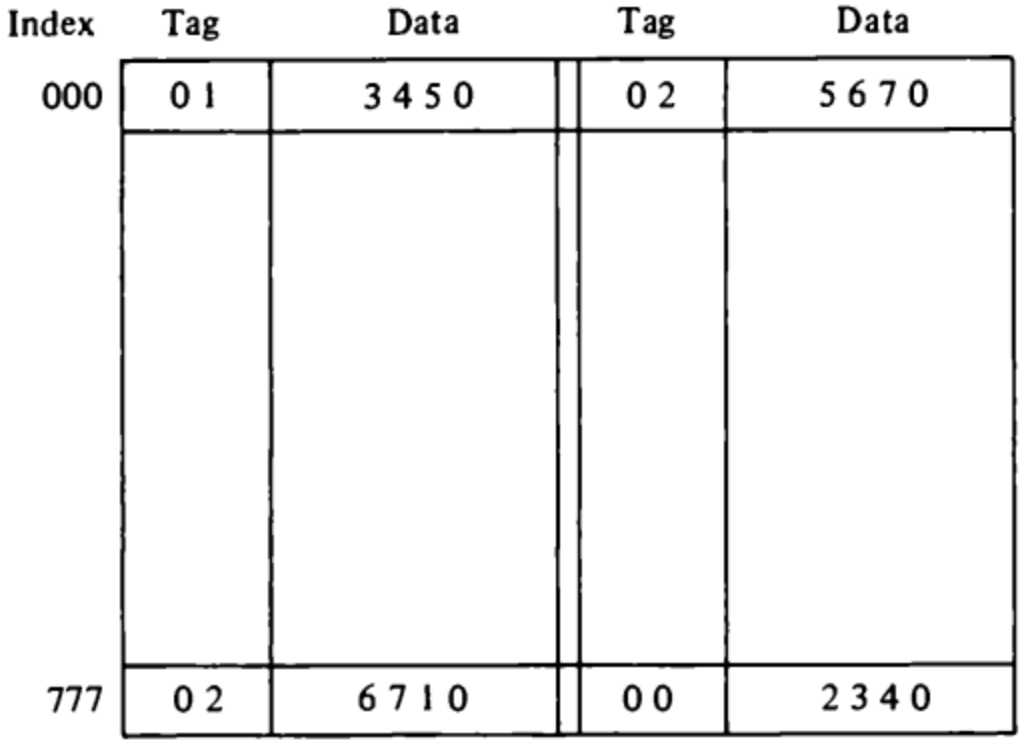


Fig. Set-Associate Mapping Cache Organization

ADITYA UNIVERSITY

Comparison of Memory Mapping Techniques

Feature	Direct Mapping	Associative Mapping	Set-Associative Mapping
Mapping Rule	One block → one fixed line	Any block → any line	One block → one set
Address Format	Tag + Index	Only Tag	Tag + Index
Flexibility	Low	Very High	Medium
Speed	Fast	Slow (parallel search)	Moderate
Hardware Cost	Low	Very High	Medium
Hit Ratio	Low	High	Moderate to High

ADITYA UNIVERSITY

VIRTUAL MEMORY

- Virtual memory is used to run large programs even when RAM is small.
- It improves memory utilization because only the required part of the program is kept in RAM.
- In virtual memory, the program is divided into small parts called **pages**.
- Only the required pages are loaded into main memory, and the remaining pages are kept in secondary memory (hard disk/SSD).
- When the CPU needs a page, it first checks whether that page is available in main memory.
- If it is available, execution continues normally.
- If it is not available, a **page fault** occurs. Then the required page is brought from secondary memory into main memory.
- **Virtual Memory = Main Memory + Secondary Memory**

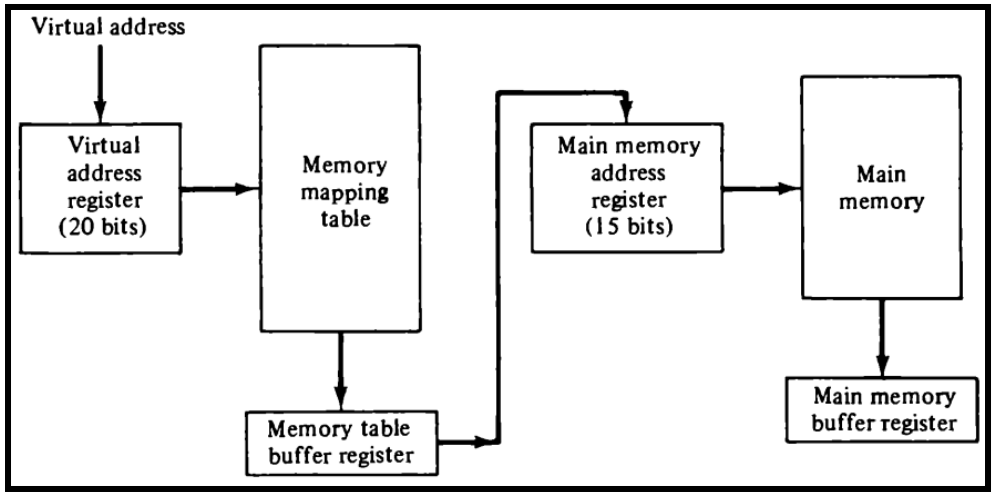
VIRTUAL MEMORY

ADITYA UNIVERSITY

Address Translation:

- The CPU generates a virtual address.
- This virtual address is translated into a physical address using a **page table**.
- Virtual address consists of: **Page Number + Offset**
- Page number is used to access the page table.
- The page table gives the corresponding **frame number** in main memory.
- The offset remains the same.
- Physical address: **Frame Number + Offset**
- Offset is the position of a byte (or word) within a page.
- It gives the exact location in the page.
- **Page Number → Roll Number**
- **Frame Number → Seat Number**

Fig. Address translation Virtual Memory



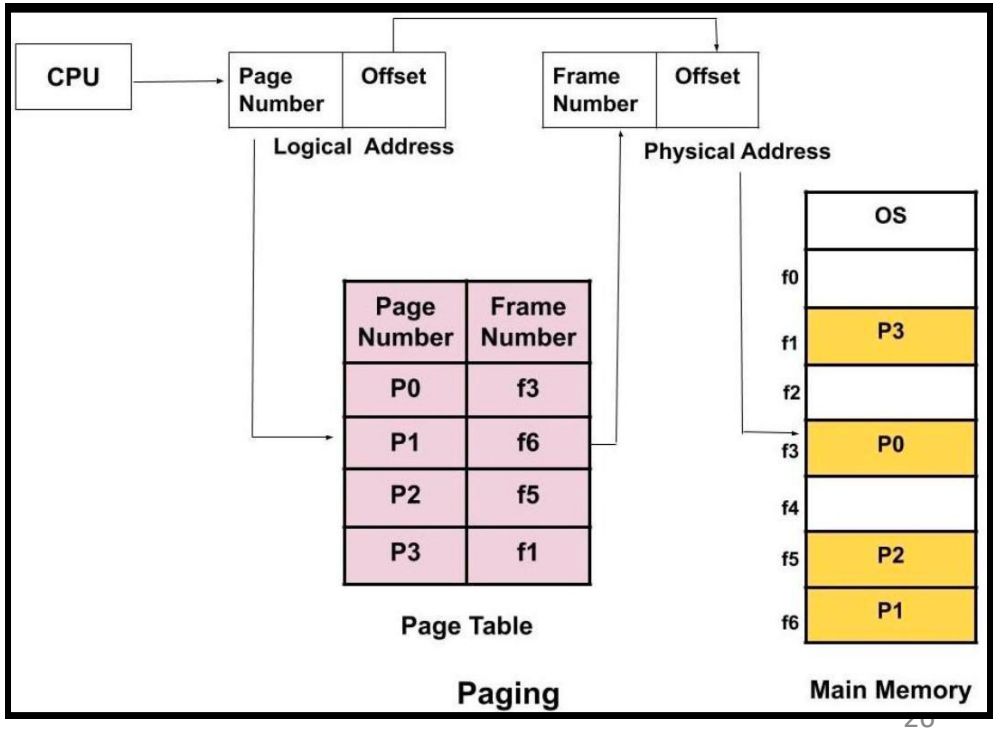


Fig. Address translation Virtual Memory

ADITYA UNIVERSITY

UNIT-4

PART-B :I/O Organization

- Peripheral Devices
- I/O interface
- Asynchronous data transfer
- Modes of transfer
  - Programmed I/O
  - Priority Interrupt
  - Direct memory access

ADITYA UNIVERSITY

Peripheral Devices

- Input or output devices that are connected to a computer are called peripheral devices.
- These devices are designed to read information into or out of the memory unit upon command from the CPU and are considered to be part of the computer system.
- For example, Keyboards, display units, and printers are common peripheral devices.
- **There are three types of peripherals:**

  1. Input peripherals: Allow user input from the outside world to the computer. Example: Keyboard, Mouse, etc.

  2. Output peripherals: Allow information output from the computer to the outside world. Example: Printer, Monitor, etc.

  3. Input-Output peripherals: Allows both input(from outside world to computer) as well as, output(from computer to the outside world). Example: Touch screen, etc.

Input-Output Interface

- An I/O interface is a hardware unit that connects the CPU with peripheral devices to enable data communication.
- It is required to overcome differences in speed, data format, and operation between the CPU and external devices.
- Processor is connected to a common I/O bus.
- I/O bus contains data, address, and control lines.
- Multiple interface modules are connected to the bus.
- Each interface is linked to a specific peripheral device (keyboard, printer, disk, tape).
- Interface handles data transfer, device selection, and control signals.
- Communication path: CPU → Bus → Interface → Device.

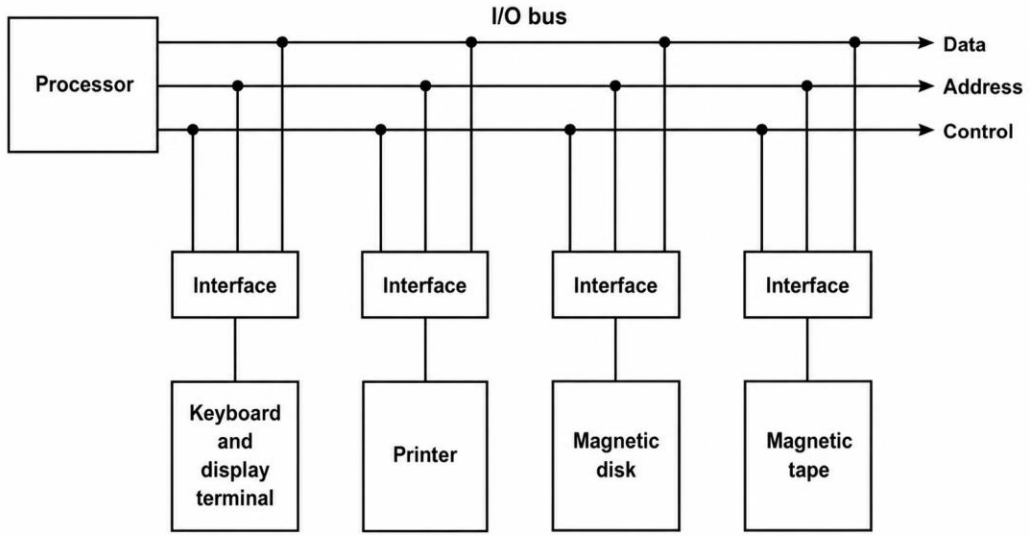


Fig: Connection of I/O bus to Input-Output devices.

Asynchronous data transfer - Strobe

- **Asynchronous data transfer** is a method where data is transferred without a common clock, using control signals for coordination.
- Used when source and destination operate at different speeds, and avoids dependency on a fixed clock timing.
- **Types of Asynchronous Transfer**

1. Strobe Control 2. Handshaking

Strobe Control:

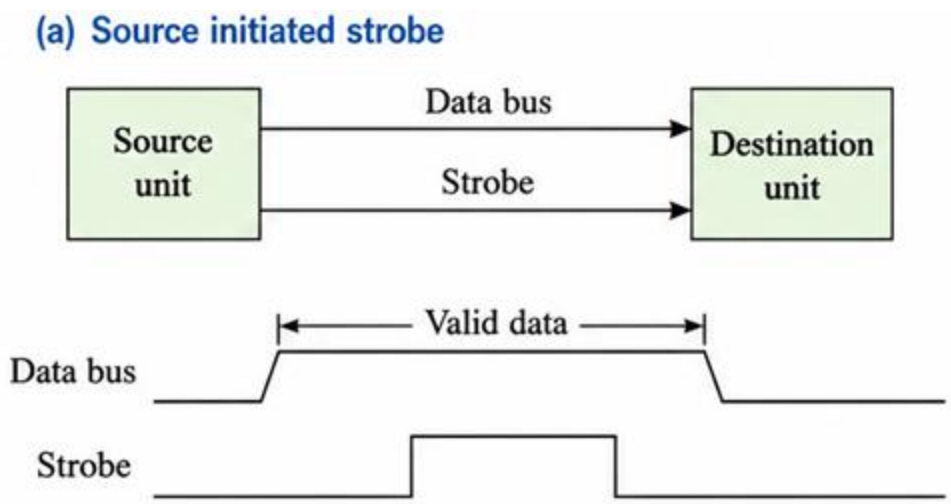
- Uses a **single control signal (Strobe)** to indicate data validity.

a) Source-Initiated Strobe

- Source places data on bus
- Source activates **strobe signal**
- Destination reads data

b) Destination-Initiated Strobe

- Destination activates **strobe** when ready
- Source places data on bus
- Data is transferred



Asynchronous data transfer - Handshaking Method

Handshaking Method

Uses two control signals for reliable transfer:

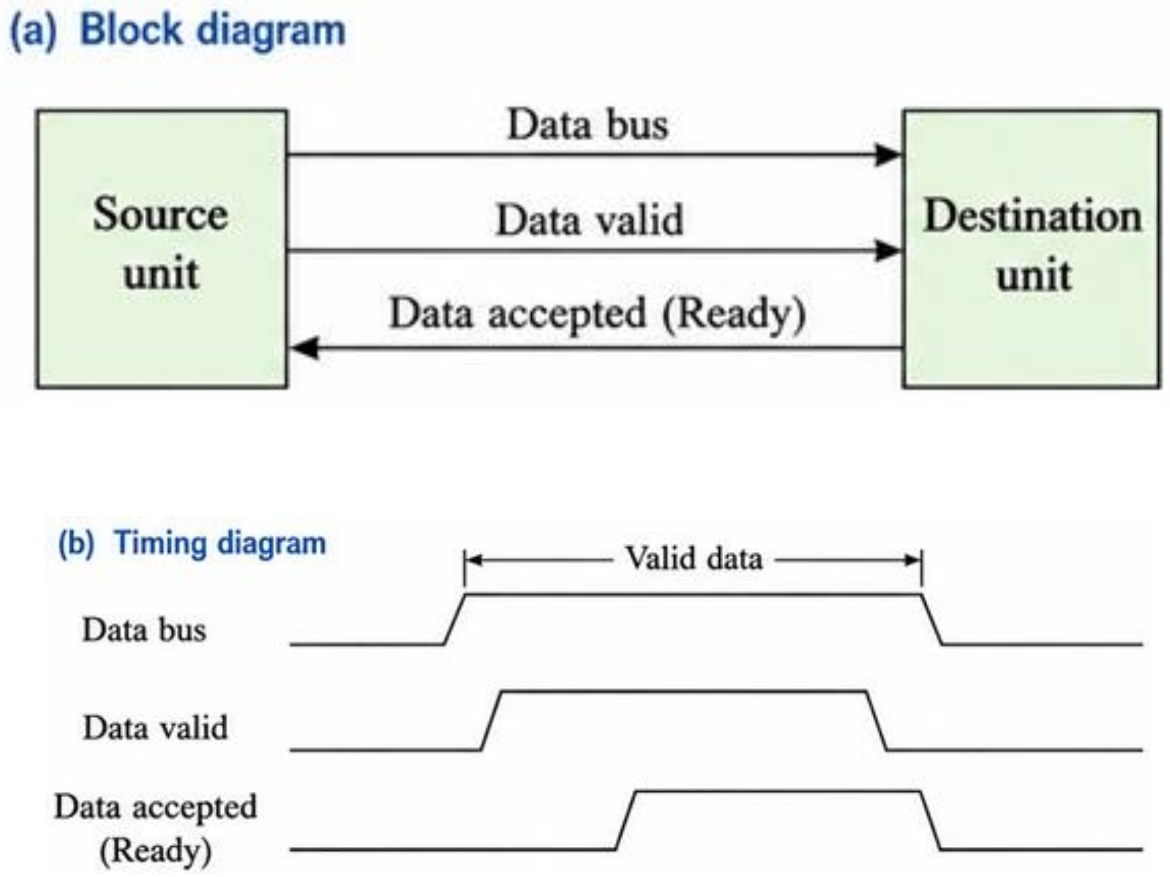
- Data Valid (from source)
- Data Accepted / Ready (from destination)

Handshaking Operation (Sequence)

- Source places data on bus
- Source enables Data Valid
- Destination checks and accepts data
- Destination sends Data Accepted / Ready signal
- Source disables Data Valid
- Destination resets signal → ready for next transfer

Advantages of Handshaking

- Ensures reliable communication
- Avoids data loss
- Suitable for devices with different speeds



Modes of Transfer

- There are three modes of data transfer:
  1. Programmed I/O
  2. Interrupt-Initiated I/O
  3. Direct Memory Access (DMA)

Programmed I/O:

- In programmed I/O, CPU continuously monitors the I/O device and controls the entire transfer operation.
- I/O device places data on I/O bus
- Device activates Data Valid signal
- Interface stores data in Data Register
- Flag bit F is set to 1
- CPU checks status register continuously
- When F = 1, CPU reads data using I/O Read signal
- Interface sends Data Accepted signal to I/O device
- Flag bit is cleared for next transfer

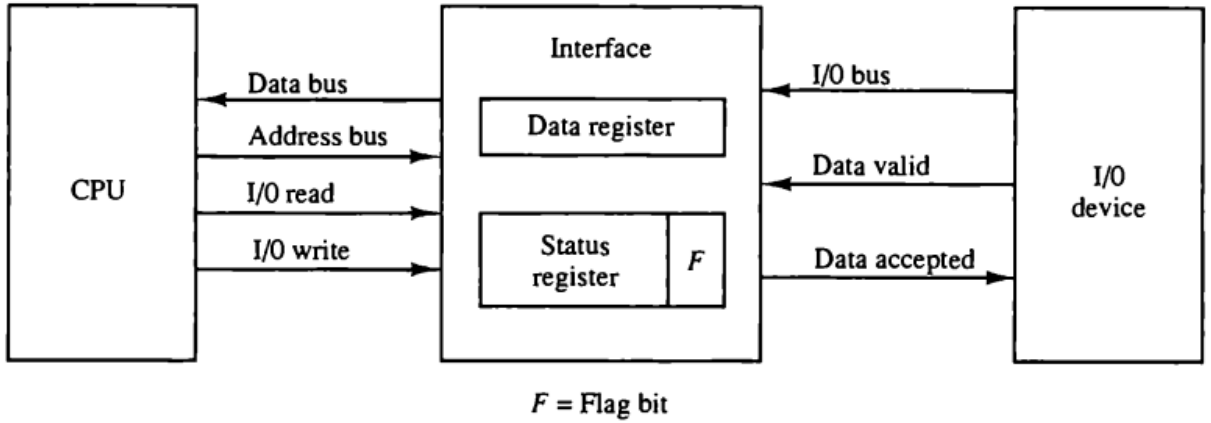


Fig: Data transfer from I/O device to CPU

Modes of Transfer

Interrupt Initiated I/O:

- In interrupt I/O, the device interrupts the CPU when it is ready for data transfer.
- CPU executes another program
- Device sends interrupt signal
- CPU stops current task temporarily
- CPU executes interrupt service routine
- After transfer, CPU returns to previous task
- CPU need not continuously monitor device

Types of Priority Interrupt:

1. Daisy-Chaining Priority Interrupt
2. Parallel Priority Interrupt

Daisy-Chaining Priority Interrupt

- The diagram shows the daisy-chaining priority interrupt system used to determine the priority among multiple I/O devices.
- Any device requiring service sends an interrupt request to CPU through the common interrupt line.
- CPU responds with an Interrupt Acknowledge (INTACK) signal.
- INTACK signal passes serially through devices:

  Device 1 → Device 2 → Device 3
- The highest-priority device captures the acknowledge signal first.
- That device places its vector address (VAD) on the processor data bus.
- Lower-priority devices do not respond once a higher-priority device is selected.

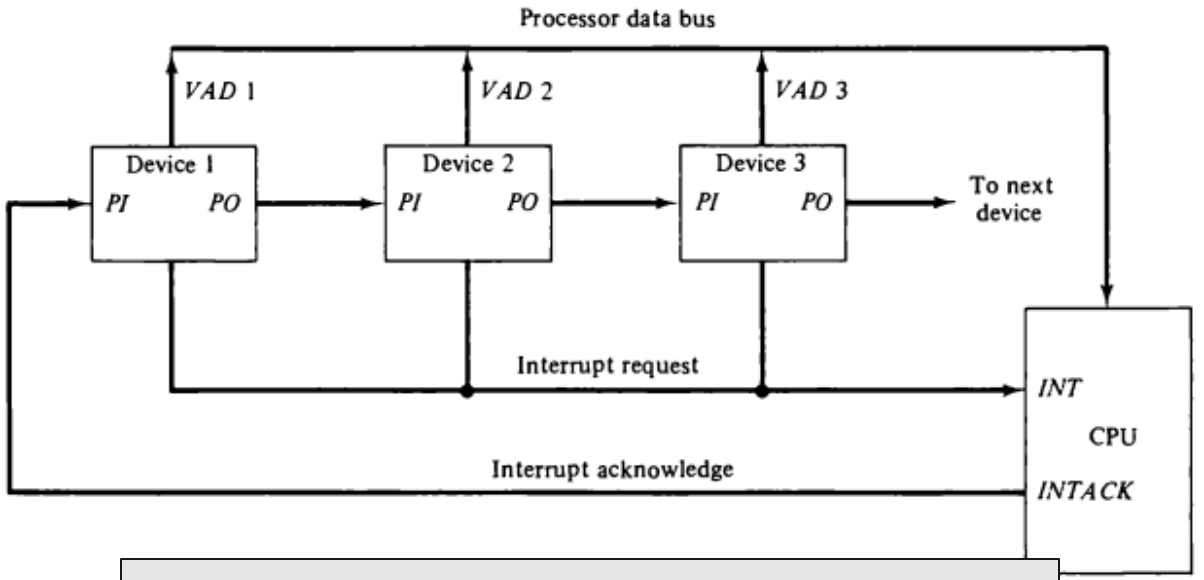


Fig: **Daisy-Chaining Priority Interrupt**

Parallel Priority Interrupt

- The diagram shows a parallel priority interrupt system used to identify and service the highest-priority interrupt request.
- Devices generate interrupt requests.
- Requests are stored in the Interrupt Register.
- Mask Register checks whether interrupts are enabled.
- Enabled requests are sent to the Priority Encoder.
- Priority Encoder selects the highest-priority request.
- Corresponding vector address (VAD) is sent to CPU.
- CPU sends INTACK signal to acknowledge interrupt.
- CPU executes the interrupt service routine.

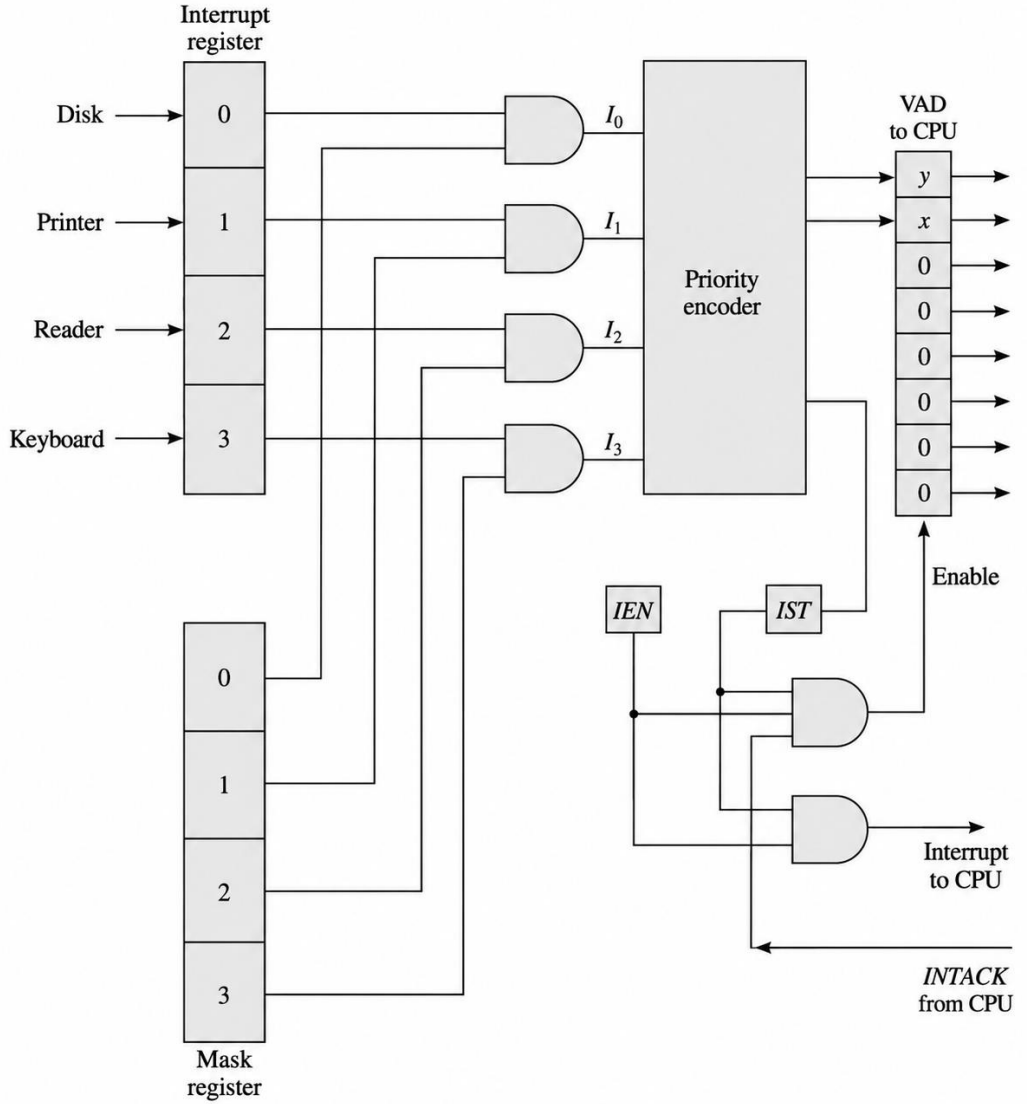


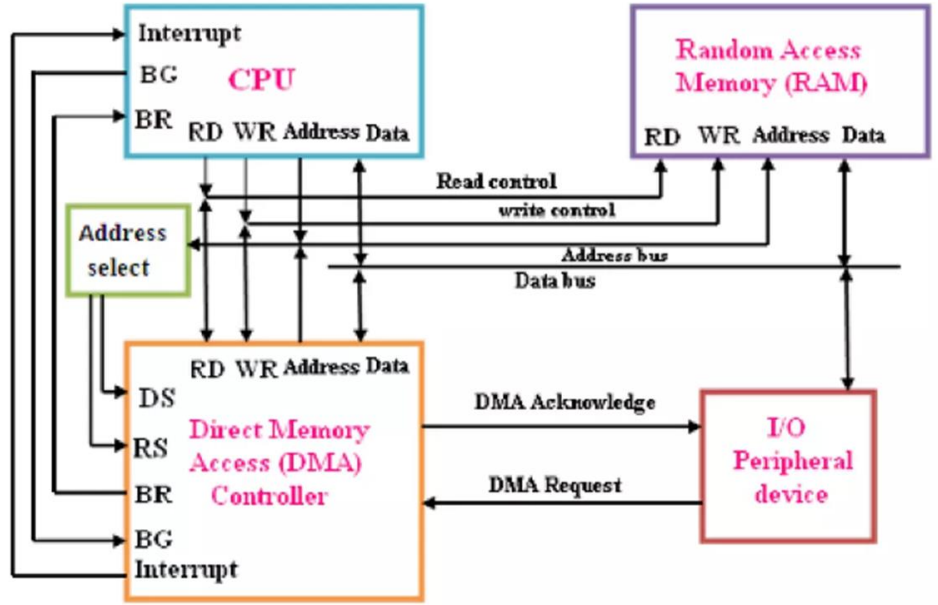
Fig: **Parallel Interrupt Hardware**

Direct Memory Access (DMA)

- DMA transfer is a method of transferring data directly between memory and I/O devices without continuous CPU involvement.
- Removing the CPU from the data transfer path improves transfer speed.
- Reduces CPU workload during large data transfers.
- The diagram shows how the DMA controller transfers data directly between I/O device and memory without continuous CPU involvement.

Working of DMA:

- I/O device sends DMA Request to DMA controller
- DMA controller sends Bus Request (BR) to CPU.
- CPU completes current operation and sends Bus Grant (BG) signal.
- DMA controller takes control of buses.
- DMA controller transfers data directly between: I/O device ↔ RAM
- CPU remains free to execute other programs
- After transfer completion, DMA controller sends Interrupt signal to CPU.CPU regains control of buses.



DMA transfer in a computer system

DMA Controller

- DMA transfer is a method of transferring data directly between memory and I/O devices without continuous CPU involvement.
- Removing the CPU from the data transfer path improves transfer speed.
- Reduces CPU workload during large data transfers.
- The diagram shows how the DMA controller transfers data directly between I/O device and memory without continuous CPU involvement.

Working of DMA Controller:

- I/O device sends DMA Request.
- DMA controller sends Bus Request (BR) to CPU.
- CPU returns Bus Grant (BG) signal.
- DMA controller takes control of buses.
- Address register provides memory address.
- Word count register tracks transfer count.
- Data transfers through data bus buffer.
- After completion, DMA controller sends interrupt to CPU.

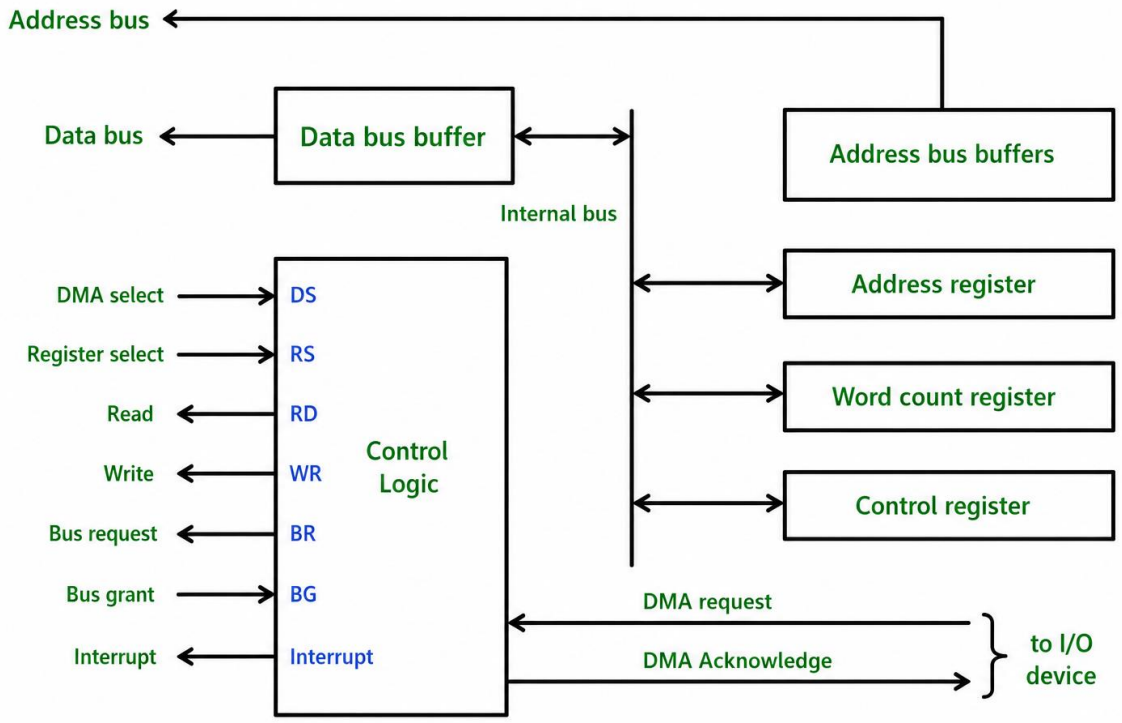


Fig: **Block diagram of DMA controller**